

Contagem paralela de objetos em matriz binaria

Sistemas Operacionais - PUCRS - 2026/II

Autores: Julia Bettiol de Oliveira e Thomaz Szeckir Gomes

1. Problema e estrategia

Entrada: matriz binaria, em que 1 representa primeiro plano e 0 representa fundo.

Objeto = componente conexo por conectividade 8: horizontal, vertical e diagonal.

Entregamos uma referencia sequencial e uma versao paralela com Pthreads.

Estrategia paralela: dividir a matriz em faixas de linhas, rotular localmente e consolidar as fronteiras.

2. Versao sequencial

Varredura linear da matriz.

Ao encontrar um 1 ainda nao visitado, incrementa a contagem e executa flood fill.

Flood fill iterativo com pilha no heap: evita estouro da pilha de execucao em matrizes grandes.

Os oito deslocamentos de vizinhanca preservam a conectividade 8.

Essa versao e a referencia de correcao e desempenho.

3. Paralelismo com Pthreads

Quantidade de threads configuravel pela linha de comando.

`pthread_create()` inicia trabalhadores e `pthread_join()` aguarda todos.

Cada thread recebe uma faixa exclusiva de linhas e realiza trabalho real de flood fill.

Matriz de entrada e somente leitura; cada thread escreve apenas nos rotulos de sua faixa.

Como as regioes de escrita sao disjuntas, nao e necessario mutex na fase local.

4. O problema das fronteiras

Somar contagens locais não funciona: um mesmo objeto pode aparecer em duas ou mais faixas.

Cada componente local recebe um rótulo globalmente único baseado no índice da célula inicial.

Na fronteira, comparamos cada célula 1 da linha superior com $c-1$, c e $c+1$ da linha inferior.

Isso preserva conexões verticais e as duas diagonais entre faixas.

5. Consolidacao com Union-Find

Depois dos `pthread_join()`, a thread principal executa a consolidacao.

Union-Find une rotulos locais que representam o mesmo objeto global.

A consolidacao ocorre sem concorrencia: nenhuma trabalhadora ainda escreve nos rotulos.

Contagem final: objetos = soma dos componentes locais - unioes efetivas (sem varrer a matriz de novo).

O encontro diagonal em divisao de regioes tambem e tratado.

6. Testes obrigatorios

Ex. 1 (5x5): esperado 3 -> sequencial 3 -> paralelo 3.

Ex. 2 (6x8): esperado 4 -> sequencial 4 -> paralelo 4.

Ex. 3 (8x8): esperado 5 -> sequencial 5 -> paralelo 5.

Ex. 4 (9x12): esperado 6 -> sequencial 6 -> paralelo 6.

Ex. 5 (12x12): esperado 7 -> sequencial 7 -> paralelo 7.

Resultados confirmados com 2, 3, 4 e 8 threads.

7. Desempenho (matriz 4000 x 4000)

Mediana de 5 repeticoes, mesma matriz.

Sequencial: 234,6 ms

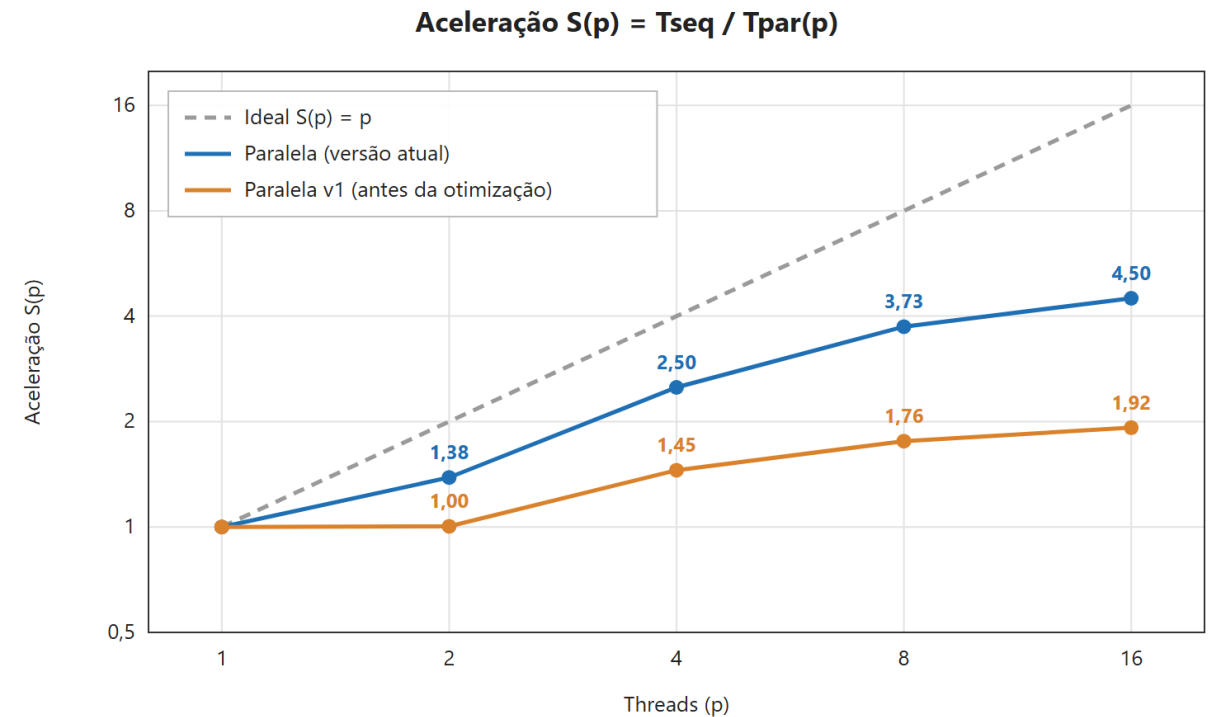
2 threads: 169,4 ms ($S = 1,38$)

4 threads: 93,7 ms ($S = 2,50$)

8 threads: 62,8 ms ($S = 3,73$)

16 threads: 52,1 ms ($S = 4,50$)

1a versao (laranja): S max. 1,92. Recontava a matriz inteira depois do join (Lei de Amdahl).



8. Matrizes pequenas x grandes

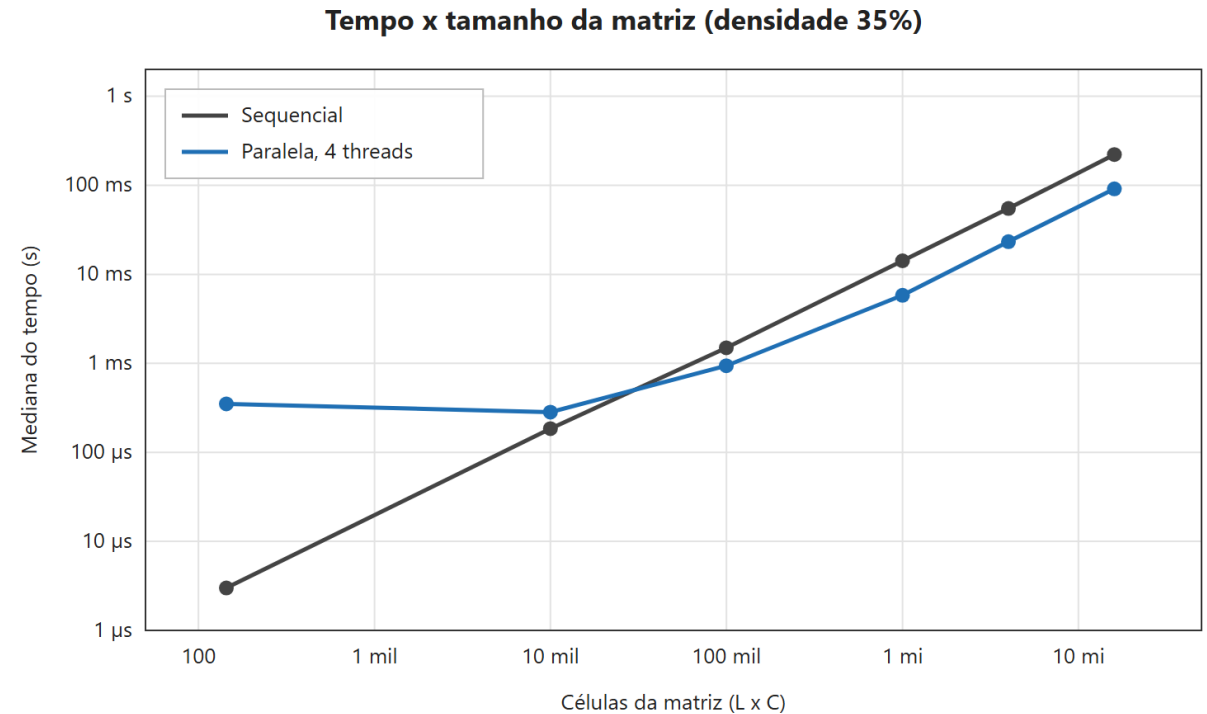
Mesma densidade, sequencial x 4 threads.

12x12 (144 células): sequencial 0,003 ms;
paralelo 0,351 ms.

Nas matrizes do enunciado o paralelo perde:
criar as threads custa mais que contar.

Empate entre 10 mil e 100 mil células.

A partir de 1 milhão de células: $S \sim 2,4$ com 4 threads.



9. Qualidade e seguranca da concorrência

ANSI C89/C90; compilacao com -Wall -Wextra -pedantic e -pthread.

Retornos de pthread_create() e pthread_join() verificados.

Alocacoes e arquivos verificados; recursos dinamicos liberados.

Sem race condition nos rotulos: faixas de escrita disjuntas.

Sem deadlock: nao ha ciclo de aquisicao de locks; consolidacao e sequencial apos join.

Valgrind: memcheck sem vazamentos; helgrind e drd sem condicoes de corrida.

10. Conclusões e demonstração

As duas versões produzem exatamente os mesmos resultados.

A decomposição distribui trabalho real entre threads.

Union-Find resolve o ponto central do trabalho: objetos que atravessam fronteiras.

Demonstração: make test; depois executar paralelo com 2 e 4 threads.

O paralelismo possui custo: aceleração depende do tamanho do problema e do ambiente.

Remover a recontagem sequencial após o join mais que dobrou a aceleração com 16 threads.